

# Custom Hooks

All hooks live in `src/hooks/`. Each hook is fully typed and follows React Query or native patterns.

## Table of Contents

1. [useNearbyGuards](#)
  2. [useCreateBooking](#)
  3. [useBookingWebSocket](#)
  4. [useWallet](#)
  5. [useRazorpay](#)
  6. [useLocation](#)
  7. [useOTP](#)
  8. [usePushNotifications](#)
  9. [useSOSTrigger](#)
- 

## 1. useNearbyGuards

`src/hooks/useNearbyGuards.ts`

```
import { useQuery } from '@tanstack/react-query';
import { guardService } from '@api/guardService';
import type { Guard } from '@types';

export function useNearbyGuards(
  latitude: number,
  longitude: number,
  radius: number = 5000,
) {
  return useQuery<Guard[]>({
    queryKey: ['nearby-guards', latitude.toFixed(4),
      longitude.toFixed(4), radius],
    queryFn: () =>
      guardService.getNearbyGuards({ latitude, longitude,
        radius }),
    enabled: latitude !== 0 && longitude !== 0,
    staleTime: 1000 * 30, // re-fetch every 30s
    refetchInterval: 1000 * 30, // auto-refresh to keep markers
      live
  })
}
```

```
});  
}
```

---

## 2. useCreateBooking

src/hooks/useCreateBooking.ts

```
import { useMutation, useQueryClient } from '@tanstack/react-  
  query';  
import { bookingService } from '@api/bookingService';  
import { useBookingStore } from '@store/useBookingStore';  
import type { CreateBookingPayload, Booking } from '@types';  
  
export function useCreateBooking() {  
  const qc = useQueryClient();  
  const setActiveBooking = useBookingStore((s) =>  
    s.setActiveBooking);  
  
  const {  
    mutateAsync: createBooking,  
    isPending: isCreating,  
    error,  
    reset,  
  } = useMutation<Booking, Error, CreateBookingPayload>({  
    mutationFn: (payload) =>  
      bookingService.createBooking(payload),  
    onSuccess: (booking) => {  
      // Store as active booking  
      setActiveBooking(booking);  
  
      // Invalidate booking list queries so they refresh  
      qc.invalidateQueries({ queryKey: ['bookings'] });  
  
      // Pre-populate detail cache  
      qc.setQueryData(['booking', String(booking.id)], booking);  
    },  
  });  
  
  return { createBooking, isCreating, error, reset };  
}
```

---

## 3. useBookingWebSocket

src/hooks/useBookingWebSocket.ts

```
import { useEffect, useRef, useState, useCallback } from 'react';  
import { useAuthStore } from '@store/useAuthStore';  
import { useBookingStore } from '@store/useBookingStore';
```

```

import { ENV } from '@lib/constants';
import type { GuardLocation, BookingStatus } from '@types';

interface WebSocketMessage {
  type:
    | 'guard.location_update'
    | 'booking.status_changed'
    | 'booking.guard_arrived'
    | 'booking.ended';
  payload: any;
}

interface UseBookingWebSocketReturn {
  guardLocation: GuardLocation | null;
  eta: number | null;
  isConnected: boolean;
}

const INITIAL_RECONNECT_DELAY = 1000;
const MAX_RECONNECT_DELAY = 30_000;
const MAX_RECONNECT_ATTEMPTS = 10;

export function useBookingWebSocket(
  bookingId: number | null,
): UseBookingWebSocketReturn {
  const token = useAuthStore((s) => s.token);
  const { updateActiveBookingStatus } = useBookingStore();

  const [guardLocation, setGuardLocation] =
    useState<GuardLocation | null>(null);
  const [eta, setEta] = useState<number | null>(null);
  const [isConnected, setIsConnected] = useState(false);

  const wsRef = useRef<WebSocket | null>(null);
  const reconnectAttempts = useRef(0);
  const reconnectTimer = useRef<ReturnType<typeof setTimeout> |
    null>(null);
  const isMounted = useRef(true);

  const getReconnectDelay = (): number =>
    Math.min(
      INITIAL_RECONNECT_DELAY * 2 ** reconnectAttempts.current,
      MAX_RECONNECT_DELAY,
    );

  const handleMessage = useCallback(
    (event: MessageEvent) => {
      let msg: WebSocketMessage;
      try {
        msg = JSON.parse(event.data as string);
      } catch {
        return;
      }
    }
  );

```

```

switch (msg.type) {
  case 'guard.location_update': {
    const { latitude, longitude, heading, updated_at } =
      msg.payload;
    setGuardLocation({
      latitude,
      longitude,
      heading: heading ?? null,
      updatedAt: updated_at,
    });
    if (msg.payload.eta_minutes !== undefined) {
      setEta(msg.payload.eta_minutes);
    }
    break;
  }

  case 'booking.status_changed': {
    const newStatus = msg.payload.status as BookingStatus;
    updateActiveBookingStatus(newStatus);
    break;
  }

  case 'booking.guard_arrived': {
    updateActiveBookingStatus('guard_arrived' as
      BookingStatus);
    setEta(0);
    break;
  }

  case 'booking.ended': {
    updateActiveBookingStatus('completed' as BookingStatus);
    wsRef.current?.close(1000, 'booking_ended');
    break;
  }

  default:
    break;
}
},
[updateActiveBookingStatus],
);

const connect = useCallback(() => {
  if (!bookingId || !token || !isMounted.current) return;
  if (reconnectAttempts.current >= MAX_RECONNECT_ATTEMPTS) {
    console.warn('[WS] Max reconnect attempts reached');
    return;
  }

  const url = `${ENV.WS_BASE_URL}/ws/booking/${bookingId}/?
    token=${token}`;
  const ws = new WebSocket(url);

```

```

wsRef.current = ws;

ws.onopen = () => {
  if (!isMounted.current) return;
  setIsConnected(true);
  reconnectAttempts.current = 0;
};

ws.onmessage = handleMessage;

ws.onerror = (e) => {
  console.warn('[WS] Error:', e);
};

ws.onclose = (e) => {
  if (!isMounted.current) return;
  setIsConnected(false);

  // Do not reconnect for clean closes or ended bookings
  if (e.code === 1000) return;

  reconnectAttempts.current += 1;
  const delay = getReconnectDelay();
  console.warn(
    `[WS] Closed (code=${e.code}). Reconnecting in ${delay}ms`
    (attempt ${reconnectAttempts.current})`,
  );

  reconnectTimer.current = setTimeout(connect, delay);
};
}, [bookingId, token, handleMessage]);

useEffect(() => {
  isMounted.current = true;
  connect();

  return () => {
    isMounted.current = false;
    reconnectTimer.current &&
      clearTimeout(reconnectTimer.current);
    wsRef.current?.close(1000, 'component_unmounted');
    wsRef.current = null;
  };
}, [connect]);

return { guardLocation, eta, isConnected };
}

```

---

## 4. useWallet

src/hooks/useWallet.ts

```
import { useQuery } from '@tanstack/react-query';
import { walletService } from '@api/walletService';
import type { Wallet, PaginatedResponse, Transaction } from '@types';

export function useWallet() {
  const walletQuery = useQuery<Wallet>({
    queryKey: ['wallet'],
    queryFn: walletService.getWallet,
    staleTime: 1000 * 60, // 1 minute
  });

  const transactionsQuery =
    useQuery<PaginatedResponse<Transaction>>({
      queryKey: ['transactions'],
      queryFn: () => walletService.getTransactions({ page: 1 }),
      staleTime: 1000 * 60,
    });

  return {
    wallet: walletQuery.data,
    transactions: transactionsQuery.data,
    isLoading: walletQuery.isLoading ||
      transactionsQuery.isLoading,
    refetch: () => {
      walletQuery.refetch();
      transactionsQuery.refetch();
    },
  };
}
```

---

## 5. useRazorpay

src/hooks/useRazorpay.ts

```
import { useState, useCallback } from 'react';
import RazorpayCheckout from 'react-native-razorpay';
import { useMutation, useQueryClient } from '@tanstack/react-query';
import { walletService } from '@api/walletService';
import { useAuthStore } from '@store/useAuthStore';
import { ENV } from '@lib/constants';
import type { ConfirmTopUpPayload } from '@types';

interface RazorpaySuccessResponse {
  razorpay_payment_id: string;
}
```

```

    razorpay_order_id: string;
    razorpay_signature: string;
  }

export function useRazorpay(initialAmount: number) {
  const [isProcessing, setIsProcessing] = useState(false);
  const user = useAuthStore((s) => s.user);
  const qc = useQueryClient();

  const { mutateAsync: confirmTopUp } = useMutation({
    mutationFn: (payload: ConfirmTopUpPayload) =>
      walletService.confirmTopUp(payload),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ['wallet'] });
      qc.invalidateQueries({ queryKey: ['transactions'] });
    },
  });

  const openCheckout = useCallback(
    async (amount: number = initialAmount) => {
      setIsProcessing(true);
      try {
        // 1. Create order on backend
        const order = await
          walletService.initiateTopUp({ amount });

        // 2. Open Razorpay checkout
        const options = {
          description: 'b-secure Wallet Top-up',
          image: 'https://bsecure.in/logo.png',
          currency: order.currency,
          key: ENV.RAZORPAY_KEY_ID,
          amount: order.amount * 100, // Razorpay expects paise
          name: 'b-secure',
          order_id: order.orderId,
          prefill: {
            email: user?.email ?? '',
            contact: user?.phone ?? '',
            name: user?.name ?? '',
          },
          theme: { color: '#3b82f6' },
        };

        const paymentData =
          (await RazorpayCheckout.open(options)) as
            RazorpaySuccessResponse;

        // 3. Confirm with backend to credit wallet
        await confirmTopUp({
          orderId: paymentData.razorpay_order_id,
          paymentId: paymentData.razorpay_payment_id,
          signature: paymentData.razorpay_signature,
        });
      }
    },
  );
}

```

```

    } catch (err: any) {
      // Razorpay returns error.code === 0 when user dismisses
      if (err?.code !== 0) {
        throw err;
      }
    } finally {
      setIsProcessing(false);
    }
  },
  [initialAmount, user, confirmTopUp],
);

return { openCheckout, isProcessing };
}

```

---

## 6. useLocation

src/hooks/useLocation.ts

```

import { useEffect, useRef } from 'react';
import * as Location from 'expo-location';
import { useLocationStore } from '@store/useLocationStore';

export function useLocation() {
  const { currentLocation, setCurrentLocation } =
    useLocationStore();
  const watchRef = useRef<Location.LocationSubscription |
    null>(null);

  useEffect(() => {
    let mounted = true;

    const startWatching = async () => {
      const { status } = await
        Location.requestForegroundPermissionsAsync();
      if (status !== 'granted' || !mounted) return;

      // Get initial position immediately
      const initial = await Location.getCurrentPositionAsync({
        accuracy: Location.Accuracy.Balanced,
      });
      if (mounted) {
        setCurrentLocation({
          latitude: initial.coords.latitude,
          longitude: initial.coords.longitude,
          accuracy: initial.coords.accuracy,
          heading: initial.coords.heading,
        });
      }

      // Watch for updates
    };
  });
}

```



```

watchRef.current = await Location.watchPositionAsync(
  {
    accuracy: Location.Accuracy.High,
    TimeInterval: 5000, // update every 5s
    distanceInterval: 10, // or every 10m
  },
  (loc) => {
    if (mounted) {
      setCurrentLocation({
        latitude: loc.coords.latitude,
        longitude: loc.coords.longitude,
        accuracy: loc.coords.accuracy,
        heading: loc.coords.heading,
      });
    }
  },
);

startWatching();

return () => {
  mounted = false;
  watchRef.current?.remove();
};
}, []);

return { currentLocation };
}

```

---

## 7. useOTP

src/hooks/useOTP.ts

```

import { useMutation } from '@tanstack/react-query';
import { authService } from '@api/authService';
import type { AuthResponse } from '@types';

export function useOTP(phone: string) {
  const {
    mutateAsync: requestOTP,
    isPending: isRequestingOTP,
  } = useMutation({
    mutationFn: () =>
      authService.requestOTP({ phone, countryCode: '+91' }),
  });

  const {
    mutateAsync: _verifyOTP,
    isPending: isVerifyingOTP,
  } = useMutation({
    mutationFn: (otp: string) =>
      authService.verifyOTP({ phone, otp, countryCode: '+91' }),
  });
}

```

```

} = useMutation<AuthResponse, Error, { otp: string;
  countryCode: string }>({
  mutationFn: ({ otp, countryCode }) =>
    authService.verifyOTP({ phone, countryCode, otp }),
});

const verifyOTP = (otp: string, countryCode: string = '+91') =>
  _verifyOTP({ otp, countryCode });

return {
  requestOTP,
  verifyOTP,
  isRequestingOTP,
  isVerifyingOTP,
};
}

```

---

## 8. usePushNotifications

src/hooks/usePushNotifications.ts

```

import { useEffect, useRef } from 'react';
import * as Notifications from 'expo-notifications';
import * as Device from 'expo-device';
import Constants from 'expo-constants';
import { Platform } from 'react-native';
import { router } from 'expo-router';
import { notificationService } from '@api/notificationService';
import { useAuthStore } from '@store/useAuthStore';

// Configure notification behaviour
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: true,
  }),
});

export function usePushNotifications() {
  const token = useAuthStore((s) => s.token);
  const notificationListener =
    useRef<Notifications.Subscription>();
  const responseListener = useRef<Notifications.Subscription>();

  useEffect(() => {
    if (!token) return;

    const registerToken = async () => {
      if (!Device.isDevice) return; // skip on simulators

```

```

const { status: existingStatus } =
  await Notifications.getPermissionsAsync();
let finalStatus = existingStatus;

if (existingStatus !== 'granted') {
  const { status } = await
    Notifications.requestPermissionsAsync();
  finalStatus = status;
}

if (finalStatus !== 'granted') return;

// Android requires a notification channel
if (Platform.OS === 'android') {
  await
    Notifications.setNotificationChannelAsync('default', {
      name: 'b-secure',
      importance: Notifications.AndroidImportance.MAX,
      vibrationPattern: [0, 250, 250, 250],
    });
}

const projectId =
  Constants.expoConfig?.extra?.eas?.projectId as string |
  undefined;
const expoPushToken = await
  Notifications.getExpoPushTokenAsync({
    projectId,
  });

// Register with backend
try {
  await
    notificationService.registerPushToken(expoPushToken.data);
} catch (err) {
  console.warn('[Push] Failed to register token', err);
}
};

registerToken();

// Foreground notification listener
notificationListener.current =
  Notifications.addNotificationReceivedListener((notification)
    => {
      console.log('[Push] Received:',
        notification.request.content.title);
    });

// Tap on notification
responseListener.current =
  Notifications.addNotificationResponseReceivedListener((response)
    => {

```

```

    const data = response.notification.request.content.data
    as {
      screen?: string;
      bookingId?: string;
    };
    if (data.screen === 'tracking') {
      router.push('/booking/tracking');
    } else if (data.bookingId) {
      router.push(`/booking/${data.bookingId}`);
    }
  });

  return () => {
    notificationListener.current?.remove();
    responseListener.current?.remove();
  };
}, [token]);
}

```

---

## 9. useSOSTrigger

src/hooks/useSOSTrigger.ts

```

import { useMutation } from '@tanstack/react-query';
import { sosService } from '@api/sosService';
import { useLocationStore } from '@store/useLocationStore';
import type { SOSTriggerPayload } from '@types';

export function useSOSTrigger(bookingId: number | null) {
  const { currentLocation } = useLocationStore();

  const {
    mutateAsync: _trigger,
    isPending: isTriggering,
    isSuccess: isTriggered,
  } = useMutation({
    mutationFn: (payload: Pick<SOSTriggerPayload, 'latitude' | 'longitude'>) => {
      if (!bookingId) throw new Error('No active booking');
      return sosService.triggerSOS({
        bookingId,
        latitude: payload.latitude,
        longitude: payload.longitude,
      });
    },
  });

  const triggerSOS = (
    coords?: { latitude: number; longitude: number },
  ) => {

```

```
    const lat = coords?.latitude ?? currentLocation?.latitude ??  
    0;  
    const lon = coords?.longitude ??  
    currentLocation?.longitude ?? 0;  
    return _trigger({ latitude: lat, longitude: lon });  
};  
  
return { triggerSOS, isTriggering, isTriggered };  
}
```